

Estructuras de Datos

Clase 6 (continuación):

Análisis de algoritmos recursivos

Dr. Sergio Alejandro Gómez

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur



8 de abril de 2024

Cálculo de tiempo de algoritmo recursivo

- 1 Determinar la entrada
- 2 Determinar el tamaño n de la entrada
- 3 Definir una recurrencia para $T(n)$
- 4 Obtener una definición no recursiva para $T(n)$
- 5 Determinar orden de tiempo de ejecución

Factorial

La función $fact(n)$ calcula $n! = 1 * 2 * 3 * 4 * \dots * (n - 1) * n$.

Recuerde que $0! = 1$

```
1:  public static int fact( int n ) {  
2:      if ( n == 0)  
3:          return 1;  
4:      else  
5:          return n * fact(n-1);  
6:  }
```

```
fact(5)  =  5 * fact(4)  
         =  5 * (4 * fact(3))  
         =  5 * (4 * (3 * fact(2)))  
         =  5 * (4 * (3 * (2 * fact(1))))  
         =  5 * (4 * (3 * (2 * (1 * fact(0)))))  
         =  5 * (4 * (3 * (2 * (1 * 1))))  
         =  5 * (4 * (3 * (2 * 1)))  
         =  5 * (4 * (3 * 2))  
         =  5 * (4 * 6)  
         =  5 * 24  
         =  120
```

Factorial

```
1:  public static int fact( int n ) {  
2:      if ( n == 0 ) cte  
3:          return 1; cte  
4:      else  
5:          return n * fact(n-1); cte +  $T_{fact}(n-1)$   
6:  }
```

① Entrada es n

② Tamaño de la entrada es n

③ Definir recurrencia para $T(n)$:

- En el caso base, testear $n = 0$ junto con retornar 1 toma tiempo c_1 , pues son dos operaciones primitivas.
- En el caso recursivo, testear $n = 0$, restar 1 a n , invocar *fact* (sin contar el tiempo que toma ejecutar *fact*($n - 1$)), multiplicar por n y retornar toma tiempo c_2 , pues son todas operaciones primitivas.

$$T(n) = \begin{cases} c_1 & \text{si } n = 0 \\ c_2 + T(n-1) & \text{si } n > 0 \end{cases}$$

$$T(N) = \begin{cases} c_1 & \text{si } N = 0 \\ c_2 + T(N - 1) & \text{si } N > 0 \end{cases}$$

- 4 Derivar una definición no recursiva para $T(n)$:

$$\begin{aligned} T(n) &= c_2 + T(n-1) &= c_2 + (c_2 + T((n-1)-1)) \\ &= 2c_2 + T(n-2) &= 2c_2 + (c_2 + T((n-2)-1)) \\ &= 3c_2 + T(n-3) &= 3c_2 + (c_2 + T((n-3)-1)) \\ &= 4c_2 + T(n-4) &= 4c_2 + (c_2 + T((n-4)-1)) \\ &= 5c_2 + T(n-5) &= \dots \\ &= ic_2 + T(n-i) & \text{(Ecuación (1))} \end{aligned}$$

Termina cuando $n - i = 0$, luego $n = i$. (Ecuación (2))

Reemplazo Ecuación (2) en Ecuación (1) y obtengo:

$$T(n) = nc_2 + T(0) = nc_2 + c_1.$$

- 5 Obtener orden de tiempo de ejecución: $T(n)$ es $\mathcal{O}(n)$.

Búsqueda binaria en arreglos ordenados

Supongamos que queremos buscar a $x = 5$ en

$a = (1, 2, 3, 5, 6, 8, 9, 10, 23, 67, 80, 92, 104)$

y $n = 13$.

$a = (1, 2, 3, 5, 6, 8, 9, 10, 23, 67, 80, 92, 104)$ con $ini=0$, $fin=12$, $medio=(0+12)/2=6$, $a[medio]=9$

|

$a = (1, 2, 3, 5, 6, 8, -, -, -, -, -, -)$ con $ini=0$, $fin=5$, $medio=(0+5)/2=2$, $a[medio]=3$

|

$a = (-, -, -, 5, 6, 8, -, -, -, -, -, -)$ con $ini=3$, $fin=5$, $medio=(3+5)/2=4$, $a[medio]=6$

|

$a = (-, -, -, 5, -, -, -, -, -, -, -)$ con $ini=3$, $fin=3$, $medio=(3+3)/2=3$, $a[medio]=5 \Rightarrow$ encontrado

Búsqueda binaria en arreglos ordenados

Peor escenario: El x a buscar no está en el arreglo a de n componentes.
Supongamos que queremos buscar a $x = 4$ en

$$a = (1, 2, 3, 5, 6, 8, 9, 10, 23, 67, 80, 92, 104)$$

y $n = 13$.

$a = (1, 2, 3, 5, 6, 8, 9, 10, 23, 67, 80, 92, 104)$ con $ini=0$, $fin=12$, $medio=(0+12)/2=6$, $a[medio]=9$

$a = (1, 2, 3, 5, 6, 8, -, -, -, -, -, -)$ con $ini=0$, $fin=5$, $medio=(0+5)/2=2$, $a[medio]=3$

$a = (-, -, -, 5, 6, 8, -, -, -, -, -, -)$ con $ini=3$, $fin=5$, $medio=(3+5)/2=4$, $a[medio]=6$

$a = (-, -, -, 5, -, -, -, -, -, -, -, -)$ con $ini=3$, $fin=3$, $medio=(3+3)/2=3$, $a[medio]=5$

$a = (-, -, -, -, -, -, -, -, -, -, -, -)$ con $ini=3$, $fin=2$ Como se cruzaron los índices, ¡ x no está!

Búsqueda binaria en arreglos ordenados

Problema: Buscar un entero x en un arreglo a de enteros ordenado de n componentes.

```
1:  public static int bSearch( int [] a, int n, int x ) {  
2:      return bSearchAux( a, 0, n-1, x );  
3:  }  
  
4:  private static int bSearchAux(int [] a, int ini, int fin, int x ) {  
5:      if( ini <= fin ) {  
6:          int medio = (ini + fin) / 2;  
7:          if( a[medio] == x ) return medio;  
8:          else if( a[medio] > x ) return bSearchAux( a, ini, medio-1, x);  
9:          else return bSearchAux( a, medio+1, fin , x);  
10:     }  
11:     else return -1;  
12: }
```


Búsqueda binaria en arreglos ordenados

1 Entrada: a

2 Tamaño de la entrada: n

3 Definición recursiva de $T_{bSearch}(n)$:

```
1:  public static int bSearch( int [] a, int n, int x ) {  
2:      return bSearchAux( a, 0, n-1, x ); cte. +  $T_{bSearchAux}(n)$   
3:  }  
  
4:  private static int bSearchAux(int [] a, int ini, int fin, int x ) {  
5:      if( ini <= fin ) { cte.  
6:          int medio = (ini + fin) / 2; cte.  
7:          if( a[medio] == x ) cte.  
7':         return medio; cte.  
8:          else if( a[medio] > x ) cte.  
8':         return bSearchAux( a, ini, medio-1, x); cte +  $T_{bSearchAux}(\frac{n-1}{2})$   
9:          else return bSearchAux( a, medio+1, fin , x); cte +  $T_{bSearchAux}(\frac{n-1}{2})$   
10:     }  
11:     else return -1; cte  
12: }
```

Búsqueda binaria en arreglos ordenados

1 Entrada: a

2 Tamaño de la entrada: n

3 Definición recursiva de $T_{bSearch}(n)$:

```
1: public static int bSearch( int [] a, int n, int x ) {  
2:     return bSearchAux( a, 0, n-1, x ); cte. +  $T_{bSearchAux}(n)$   
3: }  
  
4: private static int bSearchAux(int [] a, int ini, int fin, int x ) {  
5:     if( ini <= fin ) { cte.  
6:         int medio = (ini + fin) / 2; cte.  
7:         if( a[medio] == x ) cte.  
7':         return medio; cte.)  
8:         else if( a[medio] > x ) cte.  
8':         return bSearchAux( a, ini, medio-1, x); cte +  $T_{bSearchAux}(\frac{n-1}{2})$   
9:         else return bSearchAux( a, medio+1, fin , x); cte +  $T_{bSearchAux}(\frac{n-1}{2})$   
10:     }  
11:     else return -1; cte  
12: }
```

Como $T_{bSearch}(n) = c_0 + T_{bSearchAux}(n)$, sólo nos tenemos que preocupar de calcular $T_{bSearchAux}(n)$:

$$T(n) = \begin{cases} c_1 & \text{si } n = 0 \\ c_2 + T(\frac{n-1}{2}) & \text{si } n \geq 1 \end{cases}$$

Búsqueda binaria en arreglos ordenados

- ④ Encontrar definición no recursiva de $T(n)$:

$$T(N) = \begin{cases} c_1 & \text{si } N = 0 \\ c_2 + T(\frac{N-1}{2}) & \text{si } N \geq 1 \end{cases}$$

$$\begin{aligned} T(n) &= c_2 + T(\frac{n-1}{2}) &= c_2 + (c_2 + T(\frac{\frac{n-1}{2}-1}{2})) \\ &= 2c_2 + T(\frac{n-3}{4}) &= 2c_2 + (c_2 + T(\frac{\frac{n-3}{4}-1}{2})) \\ &= 3c_2 + T(\frac{n-7}{8}) &= 3c_2 + (c_2 + T(\frac{\frac{n-7}{8}-1}{2})) \\ &= 4c_2 + T(\frac{n-15}{16}) &= 4c_2 + (c_2 + T(\frac{\frac{n-15}{16}-1}{2})) \\ &= 5c_2 + T(\frac{n-31}{32}) &= \dots \\ &= ic_2 + T(\frac{n-(2^i-1)}{2^i}) & \text{(Ecuación 1)} \end{aligned}$$

Termina cuando $\frac{n-(2^i-1)}{2^i} = 0$, luego $n - (2^i - 1) = 0$.

Entonces, $n - 2^i + 1 = 0$; por lo tanto, $n + 1 = 2^i$ e $i = \log_2(n + 1)$ (Ecuación 2)

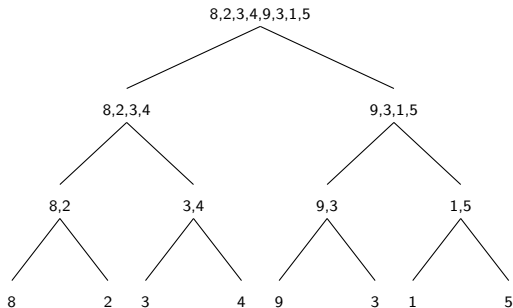
Reemplazo (2) en (1):

$$T(n) = \log_2(n + 1)c_2 + T(0) = \log_2(n + 1)c_2 + c_1$$

- ⑤ Hallar orden de tiempo de ejecución de $T(n)$: $T(n)$ es $\mathcal{O}(\log_2(n + 1))$.

MergeSort para ordenar arreglos

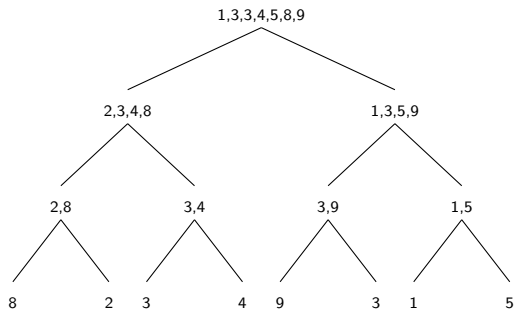
Secuencias de msort para ordenar el arreglo 8,2,3,4,9,3,1,5 (se lee de arriba hacia abajo):



En el caso base, tenemos arreglos de tamaño 1 que están trivialmente ordenados.

MergeSort para ordenar arreglos

Salidas de las recursiones haciendo merge (se lee de abajo hacia arriba):



MergeSort para ordenar arreglos

```
1:  public static void mergesort( int [] a, int n) {
2:      msort( a, 0, n-1);
3:  }

4:  private static void msort( int [] a, int ini, int fin) {
5:      if( ini < fin) {
6:          int medio = (ini + fin) / 2;
7:          msort(a, ini, medio );
8:          msort(a, medio+1, fin);
9:          merge( a, ini, medio, fin );
10:     }
11: }
```

MergeSort para ordenar arreglos

Mezcla de 2 sub-arreglos ordenados

```
1: void merge( int [] a, int ini, int medio, int fin) {
2:     int i = ini, j = medio+1, k = 0;
3:     int [] b = new int[fin-ini+1];

4:     while (i<=medio && j<=fin) {
5:         if (a[i] < a[j]) b[k++] = a[i++];
6:         else b[k++] = a[j++];
7:     }

8:     while (i<=medio) b[k++] = a[i++];
9:     while (j<=fin) b[k++] = a[j++];

10:    for(i=ini, k=0; i<=fin; i++, k++) a[i]=b[k];
11: }
```

MergeSort para ordenar arreglos

Análisis de mezcla: Escenario 1 (*i* llega a *medio* antes de que *j* llegue a *fin*)

Tamaño de entrada: n = cantidad de componentes del arreglo a con $n = fin - ini + 1$.

```
1: void merge( int [] a, int ini, int medio, int fin) {  
2:     int i = ini, j = medio+1, k = 0;  $c_1$   
3:     int [] b = new int[fin-ini+1];  $c_2$   
    $n/2 + j$  iteraciones cada una de tiempo  $c_3$   
4:     while (i<=medio && j<=fin) {  
5:         if (a[i] < a[j]) b[k++] = a[i++];  
6:         else b[k++] = a[j++];  
7:     }
```

0 iteraciones de tiempo c_4

```
8:     while (i<=medio) b[k++] = a[i++];
```

$n/2 - j$ iteraciones de tiempo c_5

```
9:     while (j<=fin) b[k++] = a[j++];
```

n iteraciones de tiempo c_6

```
10:    for(i=ini, k=0; i<=fin; i++, k++) a[i]=b[k];
```

```
11: }
```

$$T(n) = c_1 + c_2 + c_3\left(\frac{n}{2} + j\right) + c_4(0) + c_5\left(\frac{n}{2} - j\right) + c_6n \leq c_7n = \mathcal{O}(n)$$

MergeSort para ordenar arreglos

Análisis de mezcla: Escenario 2 (*j* llega a *fin* antes de que *i* llegue a *medio*)

Tamaño de entrada: $n =$ cantidad de componentes del arreglo a con $n = fin - ini + 1$.

```
1: void merge( int [] a, int ini, int medio, int fin) {  
2:     int i = ini, j = medio+1, k = 0;  $c_1$   
3:     int [] b = new int[fin-ini+1];  $c_2$   
4:     while (i<=medio && j<=fin) {  
5:         if (a[i] < a[j]) b[k++] = a[i++];  
6:         else b[k++] = a[j++];  
7:     }
```

$i + n/2$ iteraciones cada una de tiempo c_3

```
8:     while (i<=medio) b[k++] = a[i++];  
9:     while (j<=fin) b[k++] = a[j++];  
10:    for(i=ini, k=0; i<=fin; i++, k++) a[i]=b[k];  
11: }
```

$n/2 - i$ iteraciones de tiempo c_4

0 iteraciones de tiempo c_5

n iteraciones de tiempo c_6

$$T(n) = c_1 + c_2 + c_3(i + \frac{n}{2}) + c_4(\frac{n}{2} - i) + c_5(0) + c_6n \leq c_7n = \mathcal{O}(n)$$

MergeSort para ordenar arreglos

Análisis de mergesort

- 1 Entrada: a
- 2 Tamaño de la entrada: $n =$ cantidad de componentes de a
- 3 Definir $T(n)$ recursivamente:

```
1: public static void mergesort( int [] a, int n) {  
2:     msort( a, 0, n-1); cte +  $T_{msort}(n)$   
3: }  
  
4: private static void msort( int [] a, int ini, int fin) {  
5:     if( ini < fin) { cte  
6:         int medio = (ini + fin) / 2; cte  
7:         msort(a, ini, medio ); cte +  $T_{msort}(n/2)$   
8:         msort(a, medio+1, fin); cte +  $T_{msort}(n/2)$   
9:         merge( a, ini, medio, fin );  $O(n)$   
10:    }  
11: }
```

$$T(n) = \begin{cases} c_1 & \text{si } n = 1 \\ c_2 n + 2T(\frac{n}{2}) & \text{si } n > 1 \end{cases}$$

MergeSort para ordenar arreglos

Análisis de mergesort

- ④ Obtener definición de $T(n)$ no recursiva:

Recordamos que:

$$T(N) = \begin{cases} c_1 & \text{si } N = 1 \\ c_2 N + 2T(\frac{N}{2}) & \text{si } N > 1 \end{cases}$$

$$\begin{aligned} T(n) &= c_2 n + 2T(\frac{n}{2}) &= c_2 n + 2(c_2 \frac{n}{2} + 2T(\frac{\frac{n}{2}}{2})) &= c_2 n + c_2 n + 4T(\frac{n}{4}) \\ &= 2c_2 n + 4T(\frac{n}{4}) &= 2c_2 n + 4(c_2 \frac{n}{4} + 2T(\frac{\frac{n}{4}}{2})) &= 2c_2 n + c_2 n + 8T(\frac{n}{8}) \\ &= 3c_2 n + 8T(\frac{n}{8}) &= \dots \\ &= ic_2 n + 2^i T(\frac{n}{2^i}) \end{aligned}$$

Termina cuando $\frac{n}{2^i} = 1$, es decir $n = 2^i$ e $i = \log_2(n)$.

$$T(n) = \log_2(n) c_2 n + n T(1) = c_2 n \log_2(n) + n c_1$$

- ⑤ Obtener orden: $T(n) = c_2 n \log_2(n) + c_1 n$ es $\mathcal{O}(n \log_2(n))$

Método:

- 1 Determinar la entrada
- 2 Determinar el tamaño n de la entrada
- 3 Definir una recurrencia para $T(n)$
- 4 Obtener una definición no recursiva para $T(n)$
- 5 Determinar orden de tiempo de ejecución

Resultados:

- $T_{fact}(n) = \mathcal{O}(n)$
- $T_{bSearch}(n) = \mathcal{O}(\log_2(n))$
- $T_{mergeSort}(n) = \mathcal{O}(n \log_2(n))$